

POSIX Threads: Scheduling and Priorities

António Barros
June 26, 2026

1 Thread Scheduling

1.1 System Priority Range Investigation

The objective of this activity is to familiarise yourself with the system's scheduling capabilities and the valid priority ranges for different policies before implementing thread-specific configurations.

- **Task:** Write a C program that uses `sched_get_priority_min()` and `sched_get_priority_max()` to retrieve and print the valid priority ranges for the `SCHED_FIFO`, `SCHED_RR`, and `SCHED_OTHER` policies.
- **Reflective Analysis:** Compare the output values for the different policies. Why might the priority range for `SCHED_OTHER` differ significantly from real-time policies such as `SCHED_FIFO` or `SCHED_RR`?
- **Challenge:** Determine whether your program requires root privileges to query these ranges.

Note

Note on Portability: Be aware that the priority ranges are system-dependent. Always query these bounds at runtime rather than hardcoding values to ensure your code remains portable across different Linux environments.

Activity 2: Visualising Thread Preemption with `SCHED_FIFO`

The objective is to observe the kernel scheduler preempting a low-priority thread to execute a high-priority one, both running under the `SCHED_FIFO` policy.

- **Low-Priority Task (Worker):** Create a thread running under `SCHED_FIFO` with a lower priority value. It must print the character 'A' 2000 times. Use the system call `write(STDOUT_FILENO, "A", 1)`; to ensure atomic and unbuffered output. Insert a delay of 1 microsecond using `usleep(1)` between every two write operations.
- **High-Priority Task (Interrupter):** Create a thread running under `SCHED_FIFO` with a higher priority value. It should print a burst of ten '*' characters, followed by a sleep interval of 10 ms. Repeat this burst cycle 5 times.
- Use `pthread_setschedparam()` for both threads. Ensure the worker priority is strictly lower than the interrupter priority.
- If necessary, execute the program with root privileges (`sudo`) to allow the transition to real-time scheduling policies. Depending on your system configuration and user permissions (e.g., `CAP_SYS_NICE` capabilities), elevated privileges may not be required.

Note

Why use `write()`? Unlike `printf()`, `write()` is an unbuffered system call. Using buffered functions in multi-threaded environments can lead to interleaved output or delays caused by internal library locks, which might mask the true behavior of the scheduler.

Activity 3: Dynamic Priority Adjustment with `pthread_setschedprio()`

The objective is to modify the *Interrupter* thread behaviour dynamically during runtime, without changing its scheduling policy.

- **Task:** Modify your *Interrupter* thread to adjust its priority dynamically using `pthread_setschedprio()`.
- **Logic Requirements:**
 - Initialise the *Interrupter* with a priority **higher** than the *Worker* thread for the first two bursts.
 - After the second burst, call `pthread_setschedprio()` to lower the *Interrupter* priority to a level **below** that of the *Worker* thread for the remaining three bursts.
- Observe the output sequence. Does it differ from the output observed in the previous exercise? If so, why?

Note

Implementation Note Ensure you query the valid priority range for your system beforehand, as done in Activity 1. When calling `pthread_setschedprio()`, ensure the new priority value is within the allowed bounds to avoid errors.

Activity 4: Analysing Scheduler Behaviour with KernelShark

In this activity, you will perform a post-mortem analysis of your thread scheduling using system-level tracing tools. This provides objective evidence of when and why preemption occurs.

- **Trace Capture:** Run your compiled program from Activity 3 using `trace-cmd`. This tool records kernel events, specifically monitoring when threads are switched in or out of the CPU.

```
$ sudo trace-cmd record -e sched_switch -e sched_wakeup ./your_program
```
- **Data Visualisation:** Open the generated `trace.dat` file in `kernelshark` to visualise the execution timeline:

```
$ kernelshark trace.dat
```
- **Analysis Tasks:**
 - **Preemption Timing:** Locate the specific timestamps where the *Interrupter* thread preempts the *Worker*.
 - **Priority Shifts:** Observe the change in behaviour during the final three bursts of the *Interrupter*. Does the timeline confirm that the *Interrupter* is no longer preempting the *Worker* once its priority is lowered?
 - **Wakeup Latency:** Measure the delay between the *Interrupter* waking up (`sched_wakeup` event) and the actual moment it begins execution on the CPU.

Note

Advanced Insight Use the graphical interface of *KernelShark* to zoom in on the thread scheduling events. Look for the *Task ID* (TID) of your threads. By comparing these to the *CPU* lane, you can distinguish between threads running on the same core and those migrating across different cores.